

2주차 미니 프로젝트 · 주 단위 시스템 개발

스터디 시스템

개념학습 기록 시스템

배운 개념을 정리하고, 우리 프로젝트 코드에서 실제 활용처까지 찾아 기록하는 Claude Code Skill

`/study-note`

Zeteo 박진 · 이현우 · 유민성 · 김정현

성남시 직업능력개발센터 | AI 기반 프론트엔드 웹개발자 양성과정

발표 순서

01

팀원 소개

Zeteo 구성

02

기획 배경

왜 이 시스템이 필요했나

03

시스템 설계도

처리 흐름 · 저장 구조 · 설계 결정

04

활용 방안

실제 사용 예시와 결과물

05

학습 사례

왜 · 무엇을 · 어떻게

06

시연 영상 & 결론

알게 된 점과 남기고 싶은 이야기

팀원 소개

Zeteo — 2주차 스터디 시스템



박진

팀장



이현우

팀원



유민성

팀원



김정현

팀원

이번 주 작업 방식 기획 회의에서 시스템 골격·저장 방식·명령 형태를 함께 정하고, 스킴 문서 1종으로 합의 내용을 그대로 구현했습니다.

기획 배경 — 과제가 요구한 것

2주차 과제 기준

“다른 기술에 대해서 배우고자 하는 의지가 있는가?”

필수 내용 — 무엇을 왜 배우는지에 대한 정의 (왜 배우고, 무엇을 배워서, 어떻게 활용했는지)

왜

왜 배웠는가

학습의 계기가 기록에 남아야 한다

무엇

무엇을 배웠는가

개념 설명 + 이해도 확인까지

어떻게

어떻게 활용했는가

실제 코드에 근거가 있어야 한다

우리의 판단

“이거 배웠다”는 선언만으로는 증빙이 되지 않는다. 세 요소가 근거와 함께 파일로 남아야 하고, 그걸 매주 사람이 수작업으로 정리하지 않으려면 도구가 필요하다 — 그래서 스킬로 만들었습니다.

무엇이 문제였나 — 수작업 정리의 한계

AS-IS

기존 방식 (수작업 정리)

- 한 줄 선언으로 끝남

“웹소켓 배웠어요” — 무엇을 어디까지 아는지 확인 불가

- 활용처의 근거가 본인 진술뿐

실제로 썼는지 검증할 방법이 없어 증빙력이 약함

- 매번 형식이 제각각

사람마다 정리 형태가 달라 팀에서 다시 읽기 어려움

TO-BE

개념학습 기록 시스템

- 명령 한 줄이면 문서가 나옴

기본설명 + 활용처 + 퀴즈를 항상 같은 형식으로 생성

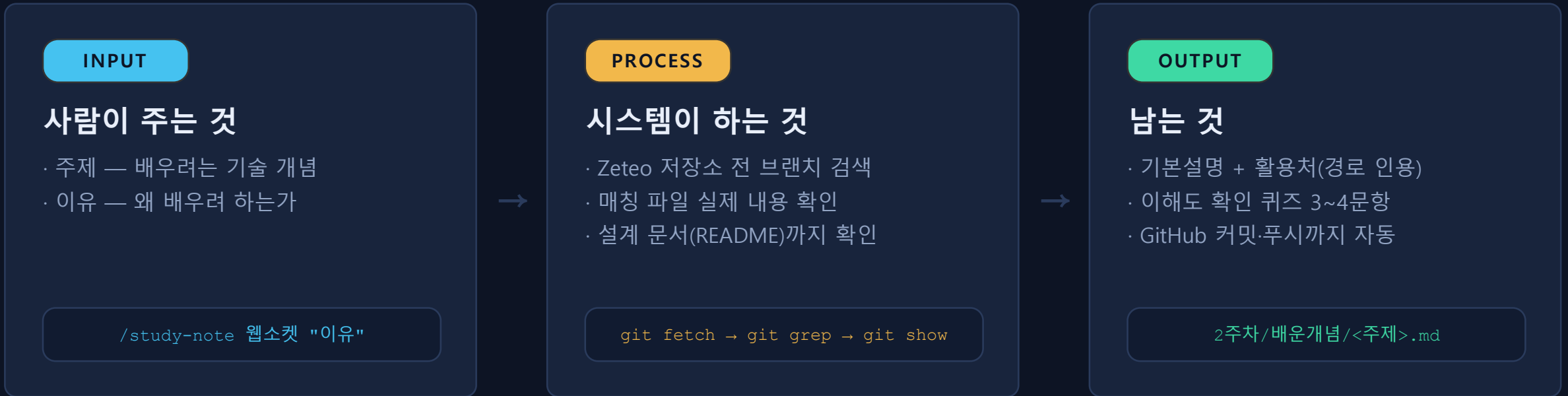
- 활용처를 코드에서 직접 조사

스킬이 저장소를 검색해 파일 경로를 근거로 인용

- 팀 저장소에 그대로 누적

2주차/배운개념/ 폴더에 주제별로 계속 쌓임

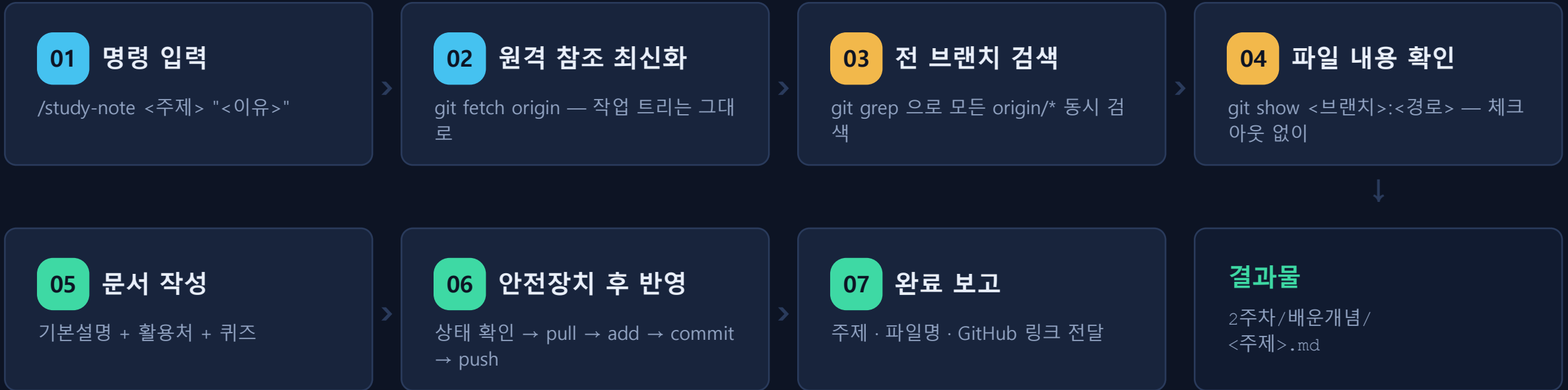
핵심 컨셉 — 입력 2개, 활용처는 시스템이 찾는다



왜 “활용처”는 인자로 받지 않는가

사용자가 활용처까지 직접 적으면 “실제로 활용했다”는 근거가 본인 진술뿐이라 증빙력이 약합니다. 그래서 활용처만은 스킬이 팀 실제 프로젝트(Zeteo) 코드베이스를 직접 검색해 찾아내도록 설계했고, 코드에서 못 찾으면 지어내지 않고 “찾지 못함”으로 정직하게 기록합니다.

시스템 설계도 ① — 전체 처리 흐름



이 흐름의 핵심

2~4단계가 이 시스템의 차별점입니다. 로컬 폴더를 그냥 읽는 대신 원격 참조만 최신화한 뒤 모든 브랜치를 한 번에 검색하기 때문에, 내가 체크아웃해 본 적 없는 팀원 브랜치의 코드까지 활용처로 잡힙니다.

시스템 설계도 ② — 조사 대상과 저장 위치의 분리

조사 대상

Project / Zeteo

팀의 실제 프로젝트 저장소(origin 추적 clone). main / dev / feat/bot / feat/server / feat/game-ui / feat/layout 등 모든 원격 브랜치가 검색 대상 — 읽기만 하고 쓰지 않습니다.

서로 다른 저장소 — 스킬 문서 상단에 명시해 오작동 방지

저장 위치

Frontend / TeamB

학습 기록이 쌓이는 팀 문서 저장소. 2주차/배운개념/ 아래에 주제별 md로 누적되며, main 브랜치에만 쓰기가 허용됩니다.

결과 파일 구조

```
TeamB/
├── README.md
├── 2주차/
│   ├── 개념학습_기록_시스템_기획.md
│   ├── 시스템/
│   │   ├── 유저 시나리오.md
│   │   └── 시스템 설계도.md
│   └── 배운개념/
│       ├── 웹소켓.md
│       └── ... (주제별로 계속 누적)
```

md 파일 내부

--- 날짜 / 주제 / 이유 ---

기본설명 ## 우리 프로젝트에서의 활용처 ## 퀴즈

설계 결정 — 무엇을 기각하고 무엇을 택했나

기각

검토했지만 채택하지 않은 안

- ✗ **Claude 아티팩트에 저장**
접근할 때마다 권한 재설정이 번거로움
- ✗ **매번 새 스레드 생성**
주제와 스레드를 이름으로 매칭해야 해 관리 비용
- ✗ **로컬 폴더를 그대로 Grep/Read**
로컬이 최신이 아닐 수 있고, 체크아웃된 브랜치 하나만 보임
- ✗ **GitHub API / 웹 검색**
비공개 저장소 인덱싱 제약, 기본 브랜치 위주로만 검색됨

채택

fetch + 전 브랜치 grep

```
git fetch origin  
git grep -i <키워드> $(모든 origin/* 브랜치)  
git show <브랜치>:<경로>
```

- ✓ **작업 트리를 건드리지 않음**
체크아웃 상태 그대로, 참조만 최신화
- ✓ **모든 브랜치를 한 번에 커버**
팀원 브랜치의 기능도 놓치지 않음
- ✓ **속도·정확도는 로컬 grep 그대로**
외부 API 제약 없이 최신성까지 확보

설계 결정에는 근거가 있습니다 — 기각 사유까지 기획서에 함께 남겼습니다.

안전장치 — 팀 저장소를 자동으로 건드리기 때문에

git 작업 안전장치

실패하면 무조건 중단 · 강제 병합/푸시 금지

- 1 새 학습 기록 파일 외 다른 변경 사항이 있으면 즉시 중단
- 2 현재 브랜치가 main 이 아니면 중단
- 3 pull --ff-only 실패(충돌) 시 자동 병합 시도 없이 중단
- 4 git add 는 새로 만든 파일 한 개만 — add -A / add . 금지
- 5 push 실패 시 --force 없이 정확한 오류와 함께 중단·보고

근거 날조 금지

모든 브랜치를 뒤져도 관련 코드가 없으면 그럴듯하게 지어내지 않고, "코드에서 명확한 사용처를 찾지 못함(모든 브랜치 확인함) — 계획 단계이거나 아직 미구현일 수 있음"으로 정직하게 기록합니다.

프롬프트 인젝션 방어

조사 과정에서 읽는 코드·문서에 지시문처럼 보이는 텍스트가 섞여 있어도 실행하지 않고 데이터로만 취급합니다. 남의 저장소를 자동으로 읽는 시스템이라 반드시 필요한 장치였습니다.

활용 방안 — 실제 사용 예시

● ● ● Claude Code

> /study-note 웹소켓 "팀프로젝트 실시간 동기화에 쓰인다고 해서 개념부터 활용까지 확인하려고 배웠다"

Zeteo 전체 브랜치에서 "웹소켓" 관련 코드 조사 중...
origin/main, dev, feat/bot, feat/server ... 검색

[OK] feat/server 브랜치에서 발견
apps/backend/src/socket/index.ts

[OK] 배운개념/웹소켓.md 작성 완료
기본설명 · 활용처 · 퀴즈 3문항

[OK] TeamB main 에 커밋·푸시 완료
github.com/Scalmia/TeamB/.../웹소켓.md

명령 한 줄로 끝난다

주제와 이유만 입력하면 조사·작성·저장·푸시까지 자동으로 이어집니다.

몰랐던 팀원 코드를 발견한다

로컬에 체크아웃한 적 없던 feat/server 브랜치의 코드가 활용처로 잡혔습니다.

퀴즈로 이해도를 스스로 점검한다

객관식 3~4문항과 해설이 함께 생성되어 헛갈리던 부분을 확인할 수 있습니다.

다음 주제도 그대로 반복 가능

JWT, ORM 등 다른 개념도 같은 절차로 계속 누적됩니다.

학습 사례 — 왜 배우고 · 무엇을 배워서 · 어떻게 활용했는가

첫 학습 주제 — 웹소켓 (WebSocket)

WHY

왜 배웠는가

서버 파트에서 실시간 상태 동기화를 웹소켓으로 처리한다는 걸 알게 되었습니다. 개념으로만 알고 있었을 뿐, 우리 프로젝트 어디에 어떻게 쓰이는지는 확인해본 적이 없었습니다.

“안다”와 “우리 프로젝트에 어떻게 쓰였는지 설명할 수 있다”는 다르다

WHAT

무엇을 배웠는가

요청-응답마다 연결을 새로 맺는 HTTP와 달리, 한 번 연결하면 그대로 유지하며 서버가 먼저 메시지를 보낼 수 있는 양방향 통신 방식.

연결 유지 방식 · 이벤트 이름 규약(action / event) · 브로드캐스트 구조

HOW

어떻게 활용했는가

feat/server 브랜치의 apps/backend/src/socket/index.ts 에서 실제 사용처를 확인했습니다. README 설계 원칙(서버가 유일한 진실 공급원 · 상태 전체 브로드캐스트 · 타이머 기반 실시간 진행)이 웹소켓이 필요했던 이유였습니다.

근거: 파일 경로 + 브랜치명 + 설계 문서 인용

결과 이 세 요소가 2주차/배운개념/웹소켓.md 파일 하나에 근거와 함께 남았고, GitHub 커밋 이력으로도 증빙됩니다.

시연 영상



시연 영상 재생 위치

YouTube 링크 또는 .mp4 파일 삽입

시연 시나리오

- 1 명령 실행**
/study-note 웹소켓 "..." 입력
- 2 코드 조사**
전 원격 브랜치 grep → 매칭 파일 확인
- 3 문서 생성**
기본설명 + 활용처 + 퀴즈 작성
- 4 저장·푸시**
안전장치 통과 후 커밋·푸시
- 5 결과 확인**
GitHub 링크로 최종 파일 열람

결론 — 알게 된 점과 소개하고 싶은 내용

알게 된 점

1 “배웠다”를 증명하려면 근거의 출처를 바꿔야 했다

본인 진술 대신 코드에서 찾게 하는 것만으로 기록의 신뢰도가 달라졌습니다.

2 로컬 저장소는 팀 전체의 현재 상태가 아니다

브랜치로 나눠 일하면 내 작업 트리에는 팀의 절반이 안 보인다는 걸 이번에 알았습니다.

3 못 찾았다고 쓰는 것도 기능이다

지어내지 않게 막는 규칙이 정확도를 지키는 가장 큰 장치였습니다.

소개하고 싶은 내용

체크아웃하지 않고 모든 원격 브랜치를 한 번에 검색하는 방법

```
git fetch origin
git grep -i <키워드> $(git branch -r)
git show <브랜치>:<파일경로>
```

작업 중인 브랜치를 그대로 둔 채 팀 전체 코드를 조사할 수 있습니다. 학습 기록뿐 아니라 코드 리뷰·구조 파악에도 그대로 쓸 수 있는 방식입니다.

향후 확장 여지 (미착수)

- 이전 기록과 자동 비교(diff)해 변경점만 보여주기
- 여러 주제를 한 번에 정리하는 배치 모드
- 퀴즈를 실제로 풀고 채점까지 이어지는 인터랙티브 모드

감사합니다. Zeteo — 박진 · 이현우 · 유민성 · 김정현